MySQL Database Lab

Creating a database and running DDL, DML and DCL queries

Duration	About 2 hours	**Name / Roll No.**	
Prerequisites	MySQL Server 8.x and MySQL Command Line Client or Workbench	**Date**	

Learning outcomes

By the end of this lab you will be able to:

- Create a database and tables using **DDL** (Data Definition Language).
- Insert, query, update and delete data using **DML** (Data Manipulation Language).
- Create users and manage their privileges using **DCL** (Data Control Language).

Category	Commands	Purpose
DDL	CREATE, ALTER, DROP, TRUNCATE, RENAME	Define or change structure
DML	INSERT, SELECT, UPDATE, DELETE	Work with data
DCL	GRANT, REVOKE	Control access
TCL	COMMIT, ROLLBACK, SAVEPOINT	Manage transactions

How to use this handout: type each statement yourself rather than pasting, and run the `DESCRIBE` or `SELECT` statements after changes to check the result. Take a screenshot of the output for each part for your lab report.

Part 0: Connect to MySQL

Open a terminal (or the MySQL Command Line Client) and log in as an administrator:

```
mysql -u root -p
```

Check that the connection works:

```
SELECT VERSION();
SHOW DATABASES;
```

Part 1: DDL (Data Definition Language)

DDL commands define and change the **structure** of the database: CREATE, ALTER, DROP, TRUNCATE, RENAME.

Step 1.1: Create and select a database

```
CREATE DATABASE college_lab;
SHOW DATABASES;
USE college_lab;
```

Step 1.2: Create tables

```
CREATE TABLE students (
    student_id   INT PRIMARY KEY AUTO_INCREMENT,
    first_name   VARCHAR(50) NOT NULL,
    last_name    VARCHAR(50) NOT NULL,
    email        VARCHAR(100) UNIQUE,
    birth_date   DATE,
    city         VARCHAR(50)
);

CREATE TABLE courses (
    course_id    INT PRIMARY KEY AUTO_INCREMENT,
    course_name  VARCHAR(100) NOT NULL,
    credits      INT CHECK (credits BETWEEN 1 AND 6)
);

CREATE TABLE enrollments (
    enrollment_id INT PRIMARY KEY AUTO_INCREMENT,
    student_id    INT NOT NULL,
    course_id     INT NOT NULL,
    grade         CHAR(2),
    FOREIGN KEY (student_id) REFERENCES students(student_id),
    FOREIGN KEY (course_id)  REFERENCES courses(course_id)
);
```

Step 1.3: Inspect the structure

```
SHOW TABLES;
DESCRIBE students;
SHOW CREATE TABLE enrollments;
```

Step 1.4: ALTER a table

```
-- Add a column
ALTER TABLE students ADD COLUMN phone VARCHAR(15);

-- Change a column's data type
ALTER TABLE students MODIFY COLUMN phone VARCHAR(20);

-- Rename a column
ALTER TABLE students RENAME COLUMN phone TO mobile;

-- Drop a column
ALTER TABLE students DROP COLUMN mobile;
```

Run `DESCRIBE students;` after each statement to see the change.

Step 1.5: Create, rename and drop a practice table

```
CREATE TABLE temp_demo (id INT, note VARCHAR(50));
RENAME TABLE temp_demo TO demo_table;
DROP TABLE demo_table;
```

Checkpoint: `SHOW TABLES;` should list only `students`, `courses` and `enrollments`.

Part 2: DML (Data Manipulation Language)

DML commands work with the **data** inside tables: INSERT, SELECT, UPDATE, DELETE.

Step 2.1: INSERT

```
INSERT INTO students (first_name, last_name, email, birth_date, city) VALUES
('Aarav',  'Sharma', 'aarav@college.edu',  '2004-03-15', 'Nagpur'),
('Priya',  'Verma',  'priya@college.edu',  '2003-11-02', 'Pune'),
('Rohan',  'Patil',  'rohan@college.edu',  '2004-07-21', 'Mumbai'),
('Sneha',  'Joshi',  'sneha@college.edu',  '2005-01-09', 'Nagpur'),
('Karan',  'Mehta',  'karan@college.edu',  '2003-09-30', 'Delhi');

INSERT INTO courses (course_name, credits) VALUES
('Database Systems', 4),
('Data Structures',  4),
('Web Technologies', 3),
('Operating Systems',3);

INSERT INTO enrollments (student_id, course_id, grade) VALUES
(1, 1, 'A'),
(1, 2, 'B'),
(2, 1, 'A'),
(3, 3, 'C'),
(4, 1, 'B'),
(4, 4, 'A'),
(5, 2, NULL);
```

Step 2.2: SELECT (querying)

```
-- All rows
SELECT * FROM students;

-- Specific columns
SELECT first_name, city FROM students;

-- Filtering
SELECT * FROM students WHERE city = 'Nagpur';

-- Multiple conditions
SELECT * FROM students WHERE city = 'Nagpur' AND birth_date > '2004-01-01';

-- Sorting and limiting
SELECT * FROM students ORDER BY birth_date DESC LIMIT 3;

-- Pattern matching
SELECT * FROM students WHERE first_name LIKE 'A%';

-- NULL check
SELECT * FROM enrollments WHERE grade IS NULL;
```

Step 2.3: Aggregates and grouping

```
SELECT COUNT(*) AS total_students FROM students;

SELECT city, COUNT(*) AS student_count
FROM students
GROUP BY city
HAVING COUNT(*) > 1;

SELECT AVG(credits) AS avg_credits FROM courses;
```

Step 2.4: JOIN

```
SELECT s.first_name, s.last_name, c.course_name, e.grade
FROM enrollments e
JOIN students s ON e.student_id = s.student_id
JOIN courses  c ON e.course_id  = c.course_id
ORDER BY s.first_name;
```

Step 2.5: UPDATE

```
UPDATE students SET city = 'Bengaluru' WHERE student_id = 3;

UPDATE enrollments SET grade = 'B' WHERE student_id = 5 AND course_id = 2;

SELECT * FROM students WHERE student_id = 3;
```

Warning: always use a `WHERE` clause with UPDATE and DELETE. Without it, every row is affected.

Step 2.6: DELETE

```
DELETE FROM enrollments WHERE enrollment_id = 7;

-- Try this and observe the error (foreign key protection):
DELETE FROM students WHERE student_id = 1;
```

Discussion question: Why did the last statement fail? How would you delete student 1 safely? (Hint: delete from `enrollments` first.)

Answer: _____

Step 2.7 (Optional): Transactions

```
START TRANSACTION;
DELETE FROM enrollments WHERE student_id = 4;
SELECT * FROM enrollments;
ROLLBACK;                       -- undo the delete
SELECT * FROM enrollments;      -- rows are back
```

Replace `ROLLBACK` with `COMMIT` to make changes permanent.

Step 2.8: DDL vs. DML, TRUNCATE

```
CREATE TABLE log_demo AS SELECT * FROM courses;
DELETE FROM log_demo;          -- DML: row by row, can be rolled back in a transaction
-- TRUNCATE TABLE log_demo;    -- DDL: removes all rows at once, cannot be rolled back
DROP TABLE log_demo;
```

Part 3: DCL (Data Control Language)

DCL commands control **access**: GRANT and REVOKE, along with user management using CREATE USER and DROP USER. You must be logged in as `root` or another user with administrative privileges.

Step 3.1: Create users

```
CREATE USER 'student_user'@'localhost' IDENTIFIED BY 'Student@123';
CREATE USER 'teacher_user'@'localhost' IDENTIFIED BY 'Teacher@123';

SELECT user, host FROM mysql.user WHERE user LIKE '%_user';
```

Step 3.2: GRANT privileges

```
-- Students: read-only access to the database
GRANT SELECT ON college_lab.* TO 'student_user'@'localhost';

-- Teacher: read, add, and change data in one table
GRANT SELECT, INSERT, UPDATE ON college_lab.enrollments TO 'teacher_user'@'localhost';
GRANT SELECT ON college_lab.students TO 'teacher_user'@'localhost';
GRANT SELECT ON college_lab.courses  TO 'teacher_user'@'localhost';

SHOW GRANTS FOR 'student_user'@'localhost';
SHOW GRANTS FOR 'teacher_user'@'localhost';
```

Step 3.3: Test the privileges

Open a **second terminal** and log in as the student:

```
mysql -u student_user -p
```

```
USE college_lab;
SELECT * FROM students;                          -- should work
INSERT INTO courses (course_name, credits)
VALUES ('AI Basics', 3);                         -- should FAIL (no INSERT privilege)
```

Record the error message in your lab report. Then log in as `teacher_user` and test an UPDATE on `enrollments` (works) and a DELETE on `enrollments` (fails).

Step 3.4: REVOKE privileges

Back in the `root` session:

```
REVOKE UPDATE ON college_lab.enrollments FROM 'teacher_user'@'localhost';
SHOW GRANTS FOR 'teacher_user'@'localhost';
```

Re-test the UPDATE as `teacher_user`. It should now fail.

Step 3.5: Clean up users

```
DROP USER 'student_user'@'localhost';
DROP USER 'teacher_user'@'localhost';
```

Part 4: Practice Exercises

Write your SQL for each exercise in your lab report.

DDL

1. Create a table `departments` (`dept_id` PK, `dept_name` NOT NULL UNIQUE).
2. Alter `students` to add a `dept_id` column and a foreign key to `departments`.

DML

3. Insert three departments and assign each student to one.
4. Display all students enrolled in "Database Systems".
5. Find the number of students per course.
6. Increase the credits of every 3-credit course by 1.
7. Delete all enrollments with a NULL grade.

DCL

8. Create a user `report_user` who can only SELECT from `courses`.
9. Grant that user SELECT on `students`, then revoke it.
10. Show the final privileges with `SHOW GRANTS`.

Part 5: Clean-up

```
DROP DATABASE college_lab;
```

Lab Report and Assessment

Submit the following:

- Screenshots of the output for each part (Parts 1 to 3).
- Your answer to the discussion question in Step 2.6.
- Your SQL and output for Exercises 1 to 10.
- The error messages you observed in Step 3.3 and after the REVOKE in Step 3.4.